# *10d.* SIMD: Independence of Iterations

Martin Alfaro
PhD in Economics

## INTRODUCTION

Broadcasting in Julia automatically leverages SIMD instructions. In contrast, whether and when for-loops will be vectorized is more subtle. Furthermore, the heuristics of the compiler, while powerful, aren't without flaws. Indeed, it's entirely possible that SIMD is implemented when it actually degrades performance, or not applied when it would've been advantageous. To address these possibilities, Julia provides the `@simd` macro to suggest the application of SIMD, giving developers a more granular control over the optimization process.

Using SIMD effectively requires understanding the conditions under which vectorization is both valid and worthwhile. Failing to meet these criteria can render SIMD infeasible or necessitate code adaptations that will slow down computations.

The ideal conditions for leveraging SIMD instructions are:

- **Independence of Iterations**: The operations in each iteration should not depend on the output of previous iterations. The only exception is reductions, which are specifically handled to ensure their feasibility.
- **Unit Strides**: the elements of each collection must be accessed sequentially.
- **No Conditional Statements**: the for-loop body should consist solely of straight-line code, avoiding branches or other conditional logic.

In this and upcoming sections, we'll elaborate on each of these items. Additionally, we'll provide guidance on how to address scenarios not conforming to them. This section in particular exclusively focuses on the independence of iterations.

> ⚠️ **Warning! - Determining Whether SIMD Has Been Implemented**
> Assessing whether SIMD instructions have been implemented requires inspecting the compiled code. Due to the complexity of this approach, we'll instead rely on execution times as a practical indicator.

## REMARKS ABOUT @SIMD IN FOR-LOOPS

Recall from the previous section that macros don't work as absolute commands. Rather, they function as optimization hints that the compiler may adopt, refine, or discard depending on the context.

In many cases, a macro simply suggests an optimization the compiler was already planning to apply, or it recommends a transformation that the compiler ultimately rejects as unsafe or inefficient. The compiler always retains the final authority over which optimizations are actually performed. In this context, adding `@simd` to a for-loop is far from a guarantee that SIMD will actually be implemented.

The challenge runs deeper still, since it's notoriously difficult to predict whether SIMD instructions are beneficial. Different CPU architectures provide different SIMD instruction sets and capabilities, and these variations can dramatically shift the performance gains you'd expect. As a result, the effectiveness of SIMD tend to vary significantly across hardware platforms. [1]

Despite this complexity, structuring operations in certain ways can improve the likelihood of implementing SIMD beneficially. By identifying these conditions, we'll be able to write code that's more amenable to SIMD optimization. It's worth remarking, though, that **the recommendations we'll present should be interpreted as general principles, rather than absolute rules**. Given ow intricate SIMD behavior can be, benchmarking remains the most reliable tool to corroborate any performance gains.

> **Safety of SIMD**
>
> Technically speaking, SIMD is a form of parallelization. We'll see in subsequent sections that parallelization can compromise code safety and cause serious errors when misused. The `@simd` macro doesn't carry these risks, since it's been designed to apply only when it's safe to do so. In particular, the compiler will disregard SIMD if the conditions for its safe application aren't met.

## INDEPENDENCE OF ITERATIONS

A condition to apply SIMD effectively in a for-loop is that its iterations are independent. This means that none of them should depend on the results of previous iterations or affect the results of subsequent ones. When this independence holds, each iteration can be executed in parallel seamlessly.

A typical scenario where iterations are independent is when transforming each element of a vector `x` using some function `foo`. In the following, we illustrate this case via a polynomial transformation of `x`. The implementation will be done via a for-loop, considering variants with and without SIMD. We'll also compare these approaches with broadcasting, which applies SIMD automatically.

Importantly, applying `@simd` in for-loops requires adding the `@inbounds` macro. Otherwise, the compiler may decide that vectorization isn't beneficial. The reason for this will be explored in a subsequent section. Basically, bounds checking introduces conditional branches, which can obstruct or entirely prevent SIMD vectorization.

**FOR-LOOP (NO SIMD)**

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    for i in eachindex(x)
        output[i] = x[i] / 2 + x[i]^2 / 3
    end

    return output
end
```

```
julia> @btime foo($x)
  783.380 μs (3 allocations: 7.629 MiB)
```

**FOR-LOOP (WITH SIMD)**

```julia
x = rand(1_000_000)

function foo(x)
    output = similar(x)

    @inbounds @simd for i in eachindex(x)
        output[i] = x[i] / 2 + x[i]^2 / 3
    end

    return output
end
```

```
julia> @btime foo($x)
  404.834 μs (3 allocations: 7.629 MiB)
```

**BROADCASTING (AUTOMATIC SIMD)**

```julia
x = rand(1_000_000)

foo(x) = @. x / 2 + x^2 / 3
```

```
julia> @btime foo($x)
  402.954 μs (3 allocations: 7.629 MiB)
```

## A SPECIAL CASE OF DEPENDENCE: REDUCTIONS

While SIMD generally requires for-loop iterations to be independent, **reductions** represent a notable exception to this rule. Even though each iteration of a reduction depends on the partial result produced by the previous one, SIMD instructions have been designed so that they can be applied safely in these scenarios.

In Julia, the application of SIMD in reductions differs depending on the data type. For integer reductions, SIMD optimization is implemented automatically. Instead, floating-point reductions require explicit intervention by annotating the for-loop with the `@simd` macro.

To illustrate this behavior, let's first consider the case of integer reductions. Because SIMD is applied automatically, you'll find that that execution times remain essentially unchanged regardless of whether you explicitly include the `@simd` macro. The following example demonstrates this in practice.

**INT64 (AUTOMATIC SIMD)**

```julia
x = rand(1:10, 10_000_000)   # random integers between 1 and 10

function foo(x)
    output = 0

    for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  3.164 ms (0 allocations: 0 bytes)
```

**INT64 (ADDING @SIMD)**

```julia
x = rand(1:10, 10_000_000)   # random integers between 1 and 10

function foo(x)
    output = 0

    @simd for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  2.835 ms (0 allocations: 0 bytes)
```

The behavior contrasts with a sum reduction consisting of floating-point operations, as shown below.

**FLOAT64 (NO AUTOMATIC SIMD)**

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  4.880 ms (0 allocations: 0 bytes)
```

**FLOAT64 (ADDING @SIMD)**

```
x = rand(10_000_000)

function foo(x)
    output = 0.0

    @simd for a in x
        output += a
    end

    return output
end
```

```
julia> @btime foo($x)
  2.805 ms (0 allocations: 0 bytes)
```

**Why Are Floating Points Treated Differently?**

**Addition of floating-point numbers doesn't obey associativity**, unlike what occurs with integers: due to the inherent imprecision of floating-point arithmetic, `(x+y)+z` may differ from `x+(y+z)`. This loss of associativity illustrates a broader point: floating-point numbers are finite-precision approximations, and therefore they don't always satisfy the mathematical properties that hold for real numbers.

The following code snippets illustrate this point.

**OPTION 1**

```
x = 0.1 + (0.2 + 0.3)
```

```
julia> x
0.6
```

**OPTION 2**

```
x = (0.1 + 0.2) + 0.3
```

```
julia> x
0.6000000000000001
```

By instructing the compiler to ignore the non-associativity of floating-point arithmetic, SIMD instructions can optimize performance by reordering terms. However, this approach assumes that the operations don't rely on a specific order of operations.

---

**FOOTNOTES**

[1.] For instance, x86 architectures (Intel and AMD processors) support SSE (Streaming SIMD Extensions) and AVX (Advanced Vector Extensions). Each of them comprises several variants that offer different vector widths and operations.